



Security Assessment



Item 21 Internal Report

September 2026

Prepared for ether.fi



Disclaimer

This document is intended for **internal** use only.



Initial Commit Hash	Final Commit Hash
29350aaf	e75116be6

Protocol Scope

- src/modules/etherfi/MidasLiquifierModule.sol

Informational Issues

I-01. Flat fees allow a compromised wallet role to grief Safes through fragmented repayments

Description: The `repay()` function charges both a proportional fee and a fixed fee for every successful repayment.

While `debtAmount` must be nonzero, there is no minimum economically meaningful repayment amount. An account holding `ETHER_FI_WALLET_ROLE` can therefore split a Safe's debt repayment into many small transactions, applying the flat fee each time.

A compromised role holder could use this behavior to repeatedly deduct payment tokens from targeted Safes while repaying only negligible amounts of debt. The griefing is bounded by the Safe's available balance, remaining debt, conversion rounding, and health-factor checks.

Recommendation: Consider enforcing a minimum repayment amount whenever a flat fee is charged

Customer's response: Acknowledged

I-02. Repayment accounting could be inconsistent in an edge case

Description: When processing a gateway-based repayment, `MidasLiquifierModule` first caps `debtAmount` against the Safe's outstanding debt and transfers that amount to the Safe:

```
uint256 debt = gateway().debtOf(user, address(debtToken));
if (debtAmount > debt) debtAmount = debt;

debtToken.safeTransfer(user, debtAmount);
return gateway().repay(user, address(debtToken), debtAmount);
```

The gateway subsequently pulls the full `debtAmount` from the Safe. If the Aave Spoke consumes less, the difference is refunded to the Safe:

```
(, uint256 assetsRepaid) = spoke.repay(reserveId, pull, safe);

if (assetsRepaid < pull) {
    IERC20(asset).safeTransfer(safe, pull - assetsRepaid);
}
```

The liquifier calculates the payment owed using only the returned `assetsRepaid`. Consequently, if the Safe's debt were reduced between the initial transfer and the Spoke repayment, the liquifier could spend the full `debtAmount` while receiving payment only for the smaller `assetsRepaid` amount. The unconsumed difference would remain in the Safe.

This is not currently exploitable through the described callback mechanism because user must be a factory-deployed `EtherFiSafe`, which does not implement token-receiver callbacks. Additionally, gateway repayments are restricted to authorized drivers, while direct Spoke repayments require an approved position manager.

Nevertheless, the accounting depends on these integration assumptions and could become unsafe if callback-enabled debt tokens, additional position managers, or other debt-mutating integrations are introduced.

Recommendation: In `_repayDebt()` in the gateway path, return the `debtAmount` that has been transferred to the user.

This recommendation applies also to the `LiquidUSDLiquifierOPModule`

Customer's response: Fixed in [PR-324](#)



Fix Review: Fix confirmed

I-03. Timelocked pair removal prevents an immediate response to compromised integrations

Description: The `removePair()` function is restricted to the Role Registry owner. Because ownership is controlled through a timelock, removing a payment-token pair cannot take effect immediately. If a configured token, redemption vault, price feed, or other pair dependency becomes compromised or behaves unexpectedly, the protocol must wait for the timelock delay before disabling the affected pair or pause the whole contract.

Recommendation: Consider allowing the fast admin role to remove a pair.

Customer's response: Acknowledged

I-04. Health checks on the Safe's approval prevent partial repayment of unhealthy legacy positions

Description: When `MidasLiquifierModule` repays a legacy Safe, it first reduces the Safe's debt through `DebtManager.repay()`.

```
debtToken.forceApprove(address(debtManager), debtAmount);  
debtManager.repay(user, address(debtToken), debtAmount);
```

Afterward, `_reclaim` instructs the Safe to approve the module to collect the corresponding Midas payment tokens:

```
data[0] = abi.encodeWithSelector(  
    IERC20.approve.selector,  
    address(this),  
    amount  
);  
  
IEtherFiSafe(user).execTransactionFromModule(to, values, data);
```

Every transaction executed through `execTransactionFromModule` invokes `EtherFiHook.postOpHook`. For legacy Safes, the hook unconditionally calls `DebtManager.ensureHealth`:

```
if (!cashModule.usesLendGateway(msg.sender)) {  
    cashModule.getDebtManager().ensureHealth(msg.sender);  
}
```

Consequently, when an unhealthy Safe makes a partial repayment that improves its position but does not completely restore it to a healthy state, the payment-token approval reverts with `AccountUnhealthy`.

Recommendation: Consider acknowledging this issue and documenting the behaviour.

Customer's response: Acknowledged



I-05. Repayment lacks a caller-supplied maximum payment amount and deadline

Description: `repay` binds the debt amount but does not bound the payment amount withdrawn from the safe. The payment and fee are calculated at execution time using live token prices and pair fee parameters, so oracle changes or fee reconfiguration between off-chain quoting and mining can cause materially more payment tokens to be extracted than expected. There is also no deadline to invalidate an outdated repayment request.

Recommendation: Add a `maxPaymentAmount` parameter covering payment plus fees and revert when the calculated amount exceeds it. Add an optional deadline and, where appropriate, a `minDebtRepaid` parameter to protect against unfavorable execution and stale requests.

Customer's response: Fixed in [PR-324](#)

Fix Review: Fix confirmed

I-06. Zero-fee gateway repayments can worsen health below the configured floor

Description: The post-repayment gateway health check is skipped when `feeAmount` is zero because the repayment is assumed to improve health. However, payment-token equivalence is calculated using `PriceProvider` while `Aave` values collateral and debt using its own oracle configuration. If these valuations differ, exchanging collateral for debt reduction can worsen the Aave health factor and place the safe below the configured minimum even with valid, nonzero oracle prices.

Recommendation: Call `_ensureGatewayFloor(user, healthFactorBefore)` after every gateway repayment, including zero-fee repayments. Its existing not-worsened comparison permits genuinely de-risking repayments below the configured floor while rejecting operations that worsen health below that floor.

This recommendation applies also to the `LiquidUSDLiquifierOPModule`

Customer's response: Fixed in [PR-324](#)

Fix Review: Fix confirmed

I-07. A position's health could be worse after a repayment

Description: `ensureMinHealthFactorNotWorsened()` enables positions to decrease their health factor as long as they remain above the configured minimum. If EtherFi's configured minimum is 1.05, a position below 1.05 should only be allowed to execute operations that improve its health factor (for example, 1.03 → 1.04).

Positions at or above 1.05 may safely perform operations that reduce health while remaining above the floor (for example, 1.08 → 1.07).

This makes sense for most DeFi operations in the other Modules (e.g deposit/withdraw/swap), however when it comes to repayments, the assumption normally is that such operations must always lead to better health factor, not worse even if above the configured minimum

Recommendation: Explicitly require that the health factor has not decreased, regardless of the minimum

Customer's response: Fixed in [PR-324](#)

Fix Review: Fix confirmed.

Note: The fix requires that the health of the position is always higher after a repayment, regardless of the minimum threshold. This may disable repayments from the MidasLiquifierModule, depending on the fee configurations and the oracle discrepancy between the PriceProvider and the Aave oracles. As a result, some partial repayments may remain blocked, depending on the fee and the amount of the repayment.



Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline. It is helpful for smart contract developers and security researchers during auditing and bug bounties.

Certora also provides services such as auditing, formal verification projects, and incident response.